

Problem 1)

The Hilbert matrix is an example of a poorly conditioned matrix, which means small changes in the matrix or the terms in a linear equation that has Hilbert matrix can lead to large changes in the solution. Hilbert matrices are used for tests in numerical analysis, to study and test numerical algorithms for solving linear equations and matrix inversion. In problems of linear regression, particularly in the context of ill-posed problems where the design matrix is poorly conditioned, like the Hilbert matrix, regularization techniques such as Tikhonov regularization (or ridge regression in statistics) are applied to stabilize the solution. The Hilbert matrix serves as a theoretical and practical example to illustrate the need for regularization methods.

N=5

```
Number of iterations to converge:6  
Vector X:
```

```
x =
```

```
1.0e+03 *  
  
0.0050  
-0.1200  
0.6300  
-1.1200  
0.6300
```

```
>> hilb(5)\ones(5,1)
```

```
ans =
```

```
1.0e+03 *  
  
0.0050  
-0.1200  
0.6300  
-1.1200  
0.6300
```

N = 8

```
Number of iterations to converge:18  
Vector X:
```

```
x =
```

```
1.0e+05 *  
  
-0.0001  
0.0050  
-0.0756  
0.4620  
-1.3860  
2.1621  
-1.6817  
0.5148
```

```
>> hilb(8)\ones(8,1)
```

```
ans =
```

```
1.0e+05 *  
  
-0.0001  
0.0050  
-0.0756  
0.4620  
-1.3860  
2.1622  
-1.6817  
0.5148
```

N = 12

```
Number of iterations to converge:116  
Vector X:
```

```
x =
```

```
1.0e+06 *  
  
-0.0000  
0.0008  
-0.0165  
0.1354  
-0.5365  
1.0260  
-0.6422  
-0.6605  
0.8059  
0.6672  
-1.2468  
0.4675
```

```
>> hilb(12)\ones(12,1)  
Warning: Matrix is close to singular
```

```
ans =
```

```
1.0e+08 *  
  
-0.0000  
0.0000  
-0.0005  
0.0083  
-0.0665  
0.3188  
-0.9617  
1.8733  
-2.3512  
1.8353  
-0.8101  
0.1544
```

N = 20

```
Number of iterations to converge:742  
Vector X:
```

```
x =
```

```
1.0e+07 *  
  
0.0000  
-0.0001  
0.0041  
-0.0486  
0.2885  
-0.8974  
1.3254  
-0.3370  
-1.1012  
0.1536  
1.1135  
0.5410  
-0.7037  
-1.1631  
-0.3256  
0.9712  
1.2695  
-0.1562  
-1.8619  
0.9281
```

```
>> hilb(20)\ones(20,1)  
Warning: Matrix is close to singular
```

```
ans =
```

```
1.0e+09 *  
  
-0.0000  
0.0000  
-0.0004  
0.0071  
-0.0592  
0.2819  
-0.7821  
1.1830  
-0.7030  
-0.1061  
-0.7593  
2.5970  
-2.4576  
1.9660  
-3.2871  
1.4698  
4.8553  
-7.6337  
4.3240  
-0.8955
```

We can see as the dimension increases, number of iterations increases, as well as the difference between the numerical results and the analytical solution (gaussian elimination) which is due to the fact that Hilbert matrix has a high condition number.

Code:

```
%% Problem 1
dimension = [5,8,12,20];
iter_max = 1000;
tol = 1e-6;
for n= dimension
    A = hilb(n);
    b = ones(n,1);
    iter = 0;
    x = zeros(n,1);
    r = b - A*x;
    p = r;
    while norm(r) > tol && iter < iter_max
        Ap = A*p;
        alpha = (p'*r)/(p'*Ap);
        x = x + alpha*p;
        r = r - alpha*Ap;
        beta = (Ap'*r)/(Ap'*p);
        p = r - beta*p;
        iter = iter + 1;
    end
    disp(['Number of iterations to converge:', num2str(iter)]);
    disp('Vector X:');
    x
end
```

Problem 2,3

#Hw 4, Mohammad Nasr, Seyed Mohammad Hosseini
problem 2)

5.2) we can prove it by contradiction:

Let's suppose that vectors $\{p_0, p_1, \dots, p_i\}$ are not linearly independent. So, there exist a set of scalars such that:

$$\beta_0 p_0 + \beta_1 p_1 + \dots + \beta_i p_i = 0, \text{ and the assumption is}$$

$$\text{also given that } p_i^T A p_j = 0 \text{ for } i \neq j$$

Now, if we multiply by $\beta_0 p_0^T A$ (or any of them) we get:

$$\beta_0^2 p_0^T A p_0 = 0 \Rightarrow \text{this means } A \text{ is not p.d. which contradicts our assumption that } A \text{ is p.d.}$$

Problem 3)

5.3) verify $\alpha_k = \frac{-r_k^T p_k}{p_k^T A p_k}$

$$\phi(x) = \frac{1}{2} x^T A x - x^T b, \quad \nabla \phi(x_k) = A x_k - b = -r_k \quad (1)$$

$$\begin{aligned} \phi(x_k + \alpha_k p_k) &= \frac{1}{2} (x_k + \alpha_k p_k)^T A (x_k + \alpha_k p_k) - (x_k + \alpha_k p_k)^T b \\ &= \frac{1}{2} x_k^T A x_k + \frac{1}{2} x_k^T A \alpha_k p_k + \frac{1}{2} \alpha_k p_k^T A x_k + \frac{1}{2} \alpha_k p_k^T A \alpha_k p_k - x_k^T b - \alpha_k p_k^T b \end{aligned}$$

$$\Rightarrow \frac{d}{d\alpha} (\phi(x_k + \alpha_k p_k)) = 0 \Rightarrow p_k^T A x_k + \alpha_k p_k^T A p_k - p_k^T b = 0$$

$$\alpha_k = \frac{p_k^T b - p_k^T A x_k}{p_k^T A p_k} \quad \left(\begin{array}{l} \text{①} \\ \Rightarrow \alpha_k = \frac{-r_k^T p_k}{p_k^T A p_k} \end{array} \right)$$

Problem 4

Results:

Number of iterations to converge:90
Vector X:1,1

Code:

```
%% Problem 4
% Parameters
alpha0 = 1;
rho = 0.5;
c = 1e-4;
tol = 1e-6;
iter = 0;
iter_max = 2000;
%Initial points
x0_1 = [1.2; 1.2];
x = x0_1;
[~, grad, ~] = rosenbrock(x);
p = grad;
while norm(grad) > tol && iter < iter_max
    alpha = backtracking_line_search(x, p, grad, alpha0, rho, c);
    x_new = x + alpha*p;
    [~, grad_new, ~] = rosenbrock(x_new);
    beta = (grad_new'*grad_new)/(grad'*grad);
    p = -grad_new + beta*p;
    grad = grad_new;
    x = x_new;
    iter = iter + 1;
end
disp(['Number of iterations to converge:', num2str(iter)]);
disp(['Vector X:', num2str(x(1)), ',', num2str(x(2))]);

function [f, grad, H] = rosenbrock(x)
a = 1; b = 100;
f = (a - x(1))^2 + b*(x(2) - x(1)^2)^2;
% Gradient of the Rosenbrock function
if nargin > 1
    grad = [-2*(a - x(1)) - 4*b*x(1)*(x(2) - x(1)^2);
            2*b*(x(2) - x(1)^2)];
end
% Hessian of the Rosenbrock function
if nargin > 2
    H = [-4*b*(x(2) - x(1)^2) + 8*b*x(1)^2 + 2, -4*b*x(1);
         -4*b*x(1), 2*b];
end
end
```

Compared to SD and Newton algorithm with line search method that we implemented in HW 2, in terms of number of iterations, it is much better than the S.D (5362), but not better than the newton's method (22 iterations). However, it is more efficient than Newton's method as it does not require hessian calculations for each iteration.

Problem 5

Number of iterations to converge:19

Vector X:1,1

Code:

```
%% Problem 5
% Parameters
alpha0 = 1;
rho = 0.9;
c = 1e-4;
tol = 1e-6;
iter = 0;
%Initial point
x0_1 = [1.2; 1.2];
x = x0_1;
[~, grad, ~] = rosenbrock(x);
p = grad;
H = eye(2); %initial guess for hessian
while norm(grad) > tol
    p = -H*grad;
    alpha = backtracking_line_search(x, p, grad, alpha0, rho, c);
    x_new = x + alpha*p;
    [~, grad_new, ~] = rosenbrock(x_new);
    s = x_new - x;
    y = grad_new - grad;
    % update hessian matrix
    H_new = (eye(2) - (s * y') / (y' * s)) * H * (eye(2) - (y * s') / (y' * s)) + ...
        (s * s') / (y' * s);
    grad = grad_new;
    x = x_new;
    H = H_new;
    iter = iter + 1;
end
disp(['Number of iterations to converge:', num2str(iter)]);
disp(['Vector X:', num2str(x(1)), ',', num2str(x(2))]);

function [f, grad, H] = rosenbrock(x)
a = 1; b = 100;
f = (a - x(1))^2 + b*(x(2) - x(1)^2)^2;
% Gradient of the Rosenbrock function
if nargin > 1
    grad = [-2*(a - x(1)) - 4*b*x(1)*(x(2) - x(1)^2);
        2*b*(x(2) - x(1)^2)];
end
% Hessian of the Rosenbrock function
if nargin > 2
```

```

        H = [-4*b*(x(2) - x(1)^2) + 8*b*x(1)^2 + 2, -4*b*x(1);
            -4*b*x(1), 2*b];
    end
end
% backtracking_algorithm
function alpha = backtracking_line_search(x, p, grad, alpha0, rho, c)
alpha = alpha0;
while rosenbrock(x + alpha*p) > rosenbrock(x) + c*alpha*grad'*p
    alpha = rho*alpha;
end
end

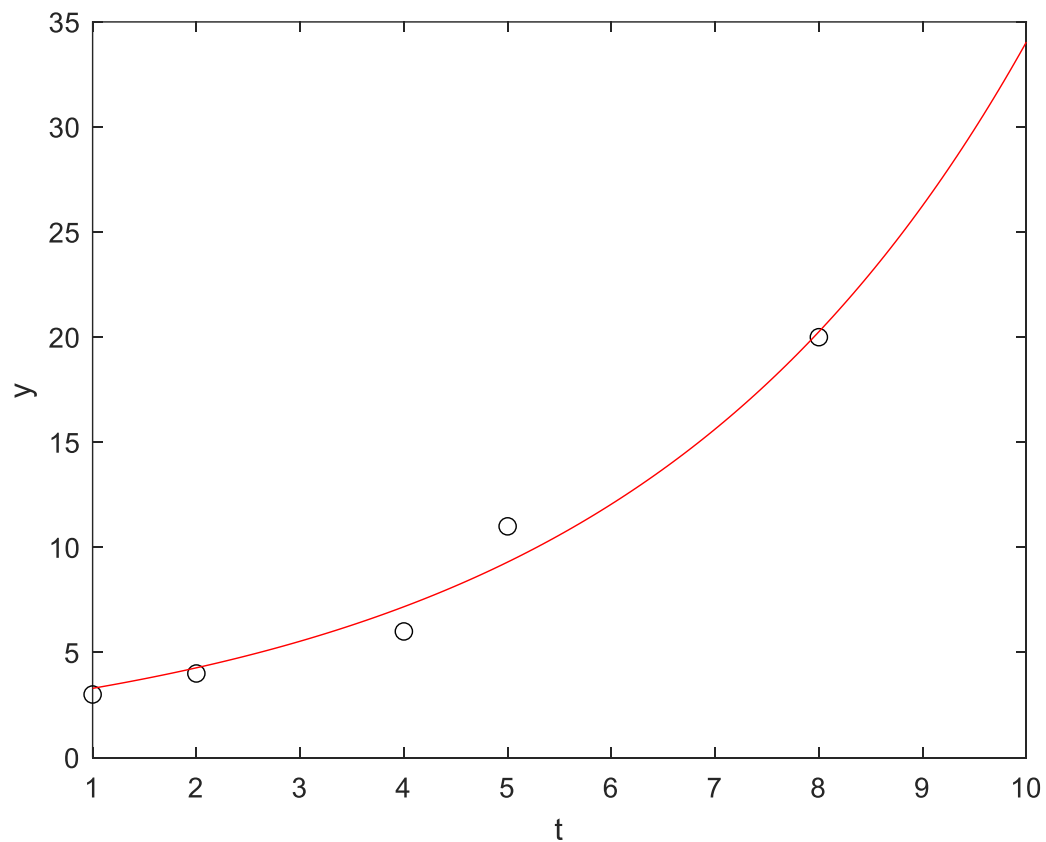
```

Compared to SD, Newton and even FR, BFGS is the most efficient one as it only takes 19 iterations to solve the problem and does not have to deal with true Hessian calculation.

Problem 6

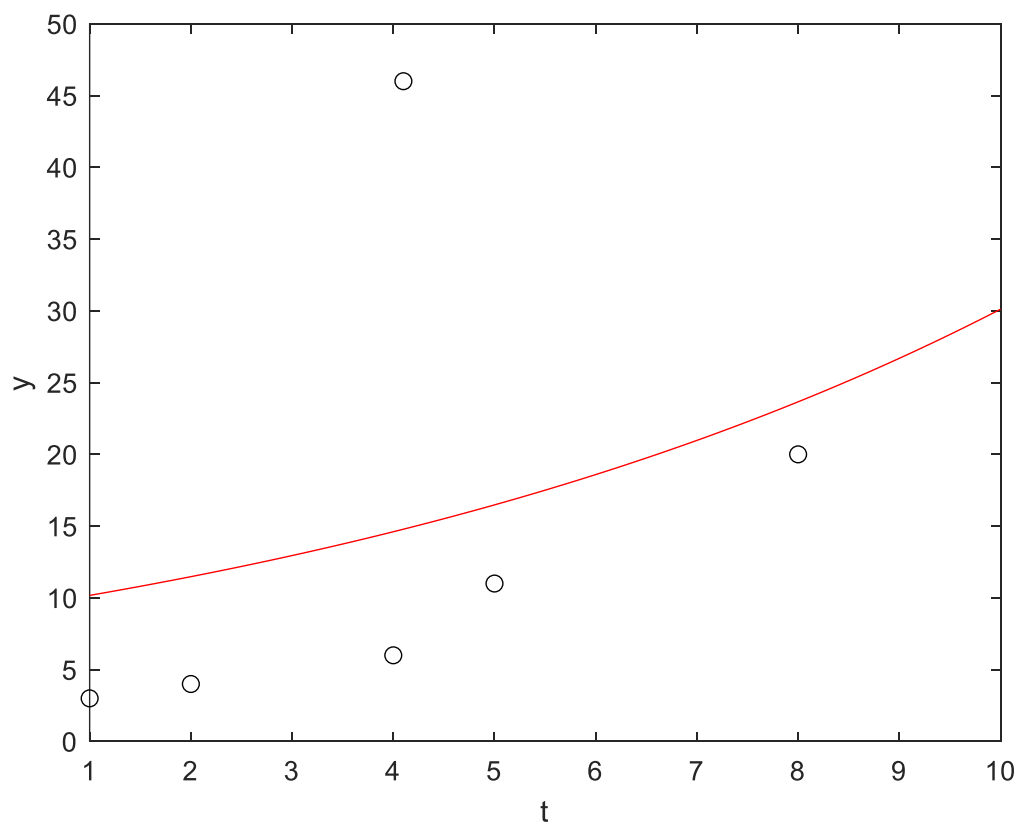
We have written our own code.

Part 1:



iter	x1	x2	grad(res)	residual
1.0e+05 *				
0.0000	0.0000	0.0000	2.7125	0.0002
2.0000	2.5416	0.2593	26.7983	4.4952
3.0000	2.5409	0.2595	0.0004	4.4943
4.0000	2.5411	0.2595	0.0000	4.4943
5.0000	2.5410	0.2595	0.0000	4.4943
6.0000	2.5410	0.2595	0.0000	4.4943

Now with the outlier and same function:

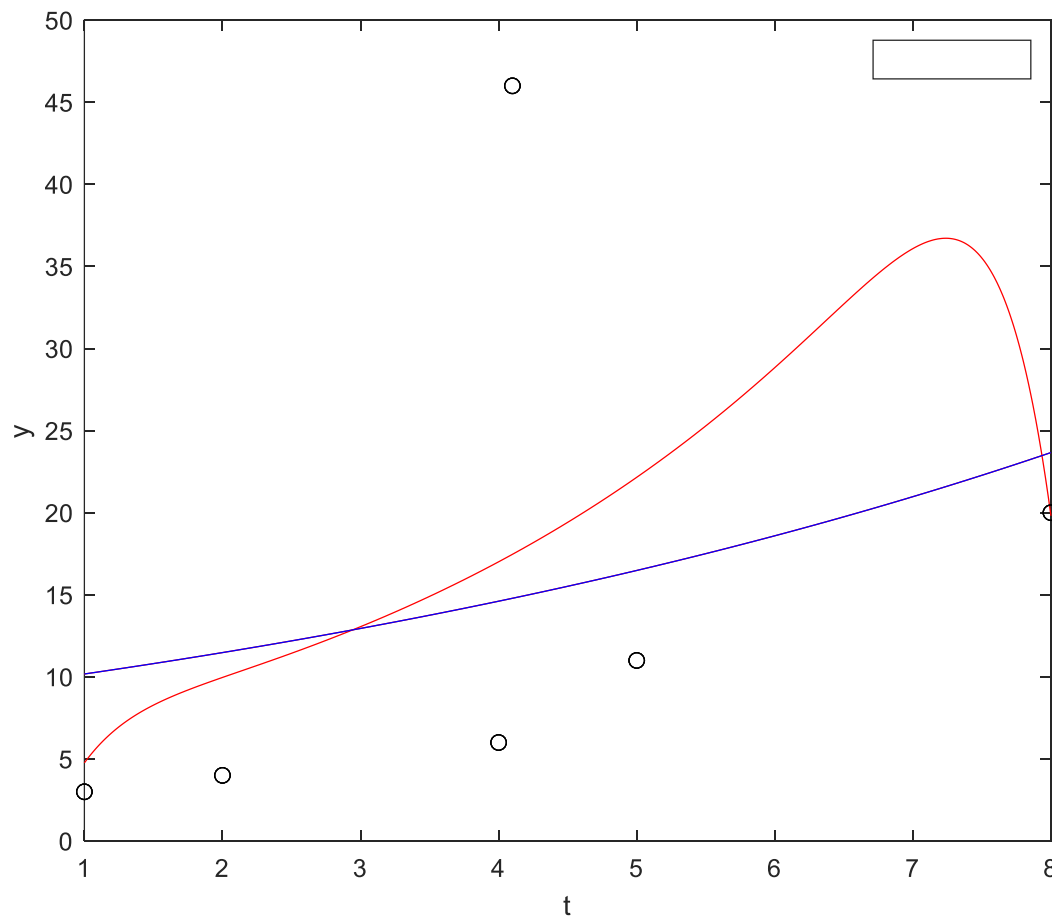


1.0e+03 *

0.0150 0.0090 0.0001 0.0000 1.1993

The number of iterations increases significantly, the residual remains high. The outlier causes the fitted function to deviate following the data trend.

Part 2



Φ might not be a good fit to the data, however, it still follows the data trend with a lower slope. On the other hand, ψ not only does not fit the data better, but also does not follow the data trend at all. Therefore, Φ is still a better fit.

Code:

```
%% problem 6
t1 = [1, 2, 4, 5, 8, 4.1]';
y = [3, 4, 6, 11, 20, 46]';
x0 = [3, 0.1]';
tol = 1e-10;
iter = 0;
iter_max = 1000;
alpha0 = 1;
rho = 0.5;
c = 1e-4;
text = sprintf('\titer\t \tx1 \tx2 \t  grad(res) \t  residual');
disp(text);
syms fit_function(x1,x2,x3,t)
fit_function(x1,x2,t) = x1*exp(x2*t);
jacobian_matrix = jacobian(fit_function, [x1, x2]);
jacobian_matrix = matlabFunction(jacobian_matrix);
f(x1,x2) = norm((fit_function(x1,x2,t1) - y), 2)^2;
x = x0;
r = double(fit_function(x(1), x(2), t1) - y);
grad = jacobian_matrix(x(1), x(2), t1)'*r;
H = jacobian_matrix(x(1), x(2), t1)*jacobian_matrix(x(1), x(2), t1);
error = norm(grad)^2;
while error > tol && iter < iter_max
    p = -H\grad;
    % backtracking_algorithm
    alpha = alpha0;
    while double(f(x(1)+alpha*p(1), x(2)+alpha*p(2))) > ...
        double(f(x(1), x(2)) + c*alpha*grad'*p)
        alpha = rho*alpha;
    end
    x_new = x + alpha*p;
    r_new = double(fit_function(x_new(1), x_new(2), t1) - y);
    grad = jacobian_matrix(x_new(1), x_new(2), t1)'*r_new;
    H = jacobian_matrix(x_new(1),
x_new(2), t1)*jacobian_matrix(x_new(1), x_new(2), t1);
    x = x_new;
    iter = iter + 1;
    error = norm(grad)^2;
    disp(double([iter x(1) x(2) double(error) norm(r_new)^2]))
end
% %plot the data and the fitted curve
figure(1);
t2 = 1:0.01:8;
plot(t1, y, 'ko', t2, fit_function(x(1), x(2), t2), 'r-'); hold on;
```

```
xlabel('t');  
ylabel('y');
```